

✓ Notebook: Estruturas LIFO (Pilha) e FIFO (Fila)

1. Exercícios em Grupo

Exercício 1 – Pilha: Simulador de Desfazer em Editor de Texto

Neste exercício, simulamos o histórico de ações de um editor de texto usando o conceito **LIFO (Last In, First Out)**.

```
class EditorDeTexto:
    def __init__(self):
        # A pilha armazenará o histórico de ações
        self.historico = []

    def executar_acao(self, acao: str):
        """Adiciona uma nova ação ao topo da pilha (push)."""
        self.historico.append(acao)
        print(f"Ação executada: '{acao}'")

    def desfazer(self):
        """Remove e reverte a última ação executada (pop)."""
        if not self.historico:
            print("Nenhuma ação para desfazer!")
            return None

        ultima_acao = self.historico.pop()
        print(f"↶ Desfazendo: '{ultima_acao}'")
        return ultima_acao

    def exibir_estado(self):
        """Exibe as ações atualmente salvas na pilha."""
        print(f"Estado atual da pilha: {self.historico}")

# --- Teste do Exercício 1 ---
editor = EditorDeTexto()

# Simulando ações do usuário
editor.executar_acao("Digitar 'Olá'")
editor.executar_acao("Digitar ' Mundo'")
editor.executar_acao("Apagar 'Mundo'")
editor.exibir_estado()

print("-" * 30)

# Desfazendo as últimas ações
```

```
# Desfazendo as últimas ações
```

```
editor.desfazer()
editor.desfazer()
editor.exibir_estado()
```

Ação executada: 'Digitar 'Olá''

Ação executada: 'Digitar ' Mundo''

Ação executada: 'Apagar 'Mundo''

Estado atual da pilha: ["Digitar 'Olá'", "Digitar ' Mundo'", "Apagar 'Mundo'"]

↺ Desfazendo: 'Apagar 'Mundo''

↺ Desfazendo: 'Digitar ' Mundo''

Estado atual da pilha: ["Digitar 'Olá'"]

✓ 📄 Exercício 2 – Fila: Sistema de Impressão de Documentos

Neste exercício, utilizamos a estrutura **FIFO (First In, First Out)** com `collections.deque` para garantir que o primeiro documento enviado seja o primeiro a ser impresso.

```
from collections import deque

class Impressora:
    def __init__(self):
        # Fila de impressão utilizando deque para performance O(1)
        self.fila_impressao = deque()

    def enviar_documento(self, nome_documento: str):
        """Adiciona um documento ao final da fila (enqueue)."""
        self.fila_impressao.append(nome_documento)
        print(f"📄 Documento '{nome_documento}' adicionado à fila.")

    def imprimir_proximo(self):
        """Remove e 'imprime' o primeiro documento da fila (dequeue)."""
        if not self.fila_impressao:
            print("Nenhum documento na fila para imprimir.")
            return None

        doc = self.fila_impressao.popleft()
        print(f"🖨️ Imprimindo: '{doc}'")
        return doc

    def exibir_fila(self):
        """Exibe todos os documentos aguardando impressão."""
        print(f"Fila atual de impressão: {list(self.fila_impressao)}")

# --- Teste do Exercício 2 ---
impressora = Impressora()
```

```

impressora.enviar_documento("Relatorio_Financeiro.pdf")
impressora.enviar_documento("Trabalho_Faculdade.docx")
impressora.enviar_documento("Boleto_Energia.pdf")

```

```

impressora.exibir_fila()
print("-" * 30)

```

```

impressora.imprimir_proximo()
impressora.imprimir_proximo()
impressora.exibir_fila()

```

```

📄 Documento 'Relatorio_Financeiro.pdf' adicionado à fila.
📄 Documento 'Trabalho_Faculdade.docx' adicionado à fila.
📄 Documento 'Boleto_Energia.pdf' adicionado à fila.
Fila atual de impressão: ['Relatorio_Financeiro.pdf', 'Trabalho_Faculdade.docx', 'Boleto_Energia.pdf']
-----
🖨️ Imprimindo: 'Relatorio_Financeiro.pdf'
🖨️ Imprimindo: 'Trabalho_Faculdade.docx'
Fila atual de impressão: ['Boleto_Energia.pdf']

```

🚀 Exercício 3 – Desafio: Fila de Atendimento Hospitalar

Neste desafio, gerenciamos uma fila em que pacientes normais entram no final (`append`) e prioritários entram no início (`appendleft`). O atendimento sempre retira do início da fila (`popleft`).

```

from collections import deque

class FilaHospital:
    def __init__(self):
        self.fila = deque()

    def adicionar_paciente(self, nome: str, prioritario: bool = False):
        """
        Pacientes normais -> Entram no final (append)
        Pacientes prioritários -> Entram no início (appendleft)
        """
        if prioritario:
            self.fila.appendleft(f"★ [PRIORIDADE] {nome}")
            print(f"🚑 Paciente Prioritário '{nome}' adicionado no INÍCIO da fila.")
        else:
            self.fila.append(f"[NORMAL] {nome}")
            print(f"👤 Paciente '{nome}' adicionado ao FINAL da fila.")

    def atender_paciente(self):
        """Atende o próximo paciente da fila (popleft)."""
        if not self.fila:

```

```

        print("Nenhum paciente aguardando atendimento.")
        return None

    paciente = self.fila.popleft()
    print(f"👨🏻‍⚕️ Atendendo agora: {paciente}")
    return paciente

    def exibir_fila(self):
        """Exibe a ordem atual da fila de espera."""
        print("\n--- Fila de Espera Atual ---")
        for idx, p in enumerate(self.fila, start=1):
            print(f"{idx}. {p}")
        print("-----\n")

# --- Teste do Desafio ---
hospital = FilaHospital()

# Chegada de pacientes
hospital.adicionar_paciente("João", prioritario=False)
hospital.adicionar_paciente("Maria", prioritario=False)
hospital.adicionar_paciente("Dona Ana (Idosa)", prioritario=True)
hospital.adicionar_paciente("Carlos", prioritario=False)
hospital.adicionar_paciente("Sr. José (Emergência)", prioritario=True)

hospital.exibir_fila()

# Atendendo os pacientes na ordem da fila
hospital.atender_paciente()
hospital.atender_paciente()
hospital.atender_paciente()

hospital.exibir_fila()

```

 Paciente 'João' adicionado ao FINAL da fila.
 Paciente 'Maria' adicionado ao FINAL da fila.
 Paciente Prioritário 'Dona Ana (Idosa)' adicionado no INÍCIO da fila.
 Paciente 'Carlos' adicionado ao FINAL da fila.
 Paciente Prioritário 'Sr. José (Emergência)' adicionado no INÍCIO da fila.

```

--- Fila de Espera Atual ---
1. ★ [PRIORIDADE] Sr. José (Emergência)
2. ★ [PRIORIDADE] Dona Ana (Idosa)
3. [NORMAL] João
4. [NORMAL] Maria
5. [NORMAL] Carlos
-----

```

 Atendendo agora: ★ [PRIORIDADE] Sr. José (Emergência)
 Atendendo agora: ★ [PRIORIDADE] Dona Ana (Idosa)
 Atendendo agora: [NORMAL] João

```

--- Fila de Espera Atual ---
1. [NORMAL] Maria
2. [NORMAL] Carlos
-----

```

✓ 2. Exercícios Complementares em Python

Pilha de Números Inteiros (Soma dos Elementos)

```

class PilhaNumerica:
    def __init__(self):
        self.pilha = []

    def push(self, valor: int):
        self.pilha.append(valor)

    def pop(self):
        if not self.pilha:
            return None
        return self.pilha.pop()

    def somar_elementos(self) -> int:
        """Retorna a soma de todos os elementos contidos na pilha."""
        return sum(self.pilha)

    def exibir(self):
        print("Pilha de números:", self.pilha)

# --- Teste ---
p = PilhaNumerica()
p.push(10)
p.push(25)
p.push(5)
p.push(40)

p.exibir()
print("Soma total dos elementos da pilha:", p.somar_elementos())

Pilha de números: [10, 25, 5, 40]
Soma total dos elementos da pilha: 80

```

✓ 💡 Resumo Teórico & quando escolher

Estrutura	Comportamento	Principais Operações	Exemplo Real
Pilha (Stack)	LIFO (<i>Last In, First Out</i>)	<code>push()</code> (inserir), <code>pop()</code> (remover)	Botão Voltar do navegador, Histórico de Ações (Ctrl+Z), Pilha de chamadas de função.
Fila (Queue)	FIFO (<i>First In, First Out</i>)	<code>enqueue()</code> (inserir), <code>dequeue()</code> (remover)	Fila de impressão, chamadas em call center, processamento de tarefas em background.

3 Fila de Atendimento de Clientes

```
from collections import deque

class FilaAtendimento:
    def __init__(self):
        self.fila = deque()

    def chegar_cliente(self, nome: str):
        self.fila.append(nome)
        print(f"Cliente '{nome}' entrou na fila.")

    def atender_cliente(self):
        if not self.fila:
            print("Fila vazia.")
            return
        cliente = self.fila.popleft()
        print(f"Atendendo cliente: {cliente}")

    def exibir_fila(self):
        print("Clientes aguardando:", list(self.fila))

# --- Teste ---
f = FilaAtendimento()
f.chegar_cliente("Lucas")
f.chegar_cliente("Beatriz")
f.chegar_cliente("Gabriel")

f.exibir_fila()
f.atender_cliente()
f.exibir_fila()
```

3. Exercícios Complementares em C

Pilha em C (Array Estático)

```
# include <stdio.h>
# include <stdlib.h>
```

```
# include <stdbool.h>

# define CAPACIDADE 5

typedef struct {
    int itens[CAPACIDADE];
    int topo;
} Pilha;

// Inicializa a pilha com o topo em -1 (vazia)
void inicializar(Pilha *p) {
    p->topo = -1;
}

bool estaCheia(Pilha *p) {
    return p->topo == CAPACIDADE - 1;
}

bool estaVazia(Pilha *p) {
    return p->topo == -1;
}

// Inserção na pilha (push)
void push(Pilha *p, int valor) {
    if (estaCheia(p)) {
        printf("Erro: Pilha cheia (Overflow)!\n");
        return;
    }
    p->topo++;
    p->itens[p->topo] = valor;
    printf("Inserido %d na pilha.\n", valor);
}

// Remoção da pilha (pop)
int pop(Pilha *p) {
    if (estaVazia(p)) {
        printf("Erro: Pilha vazia (Underflow)!\n");
        return -1;
    }
    int valor = p->itens[p->topo];
    p->topo--;
    return valor;
}
```

```

}

void exibir(Pilha *p) {
    if (estaVazia(p)) {
        printf("Pilha vazia.\n");
        return;
    }
    printf("Conteudo da pilha (topo para base): ");
    for (int i = p->topo; i >= 0; i--) {
        printf("%d ", p->itens[i]);
    }
    printf("\n");
}

int main() {
    Pilha p;
    inicializar(&p);

    push(&p, 10);
    push(&p, 20);
    push(&p, 30);

    exibir(&p);

    printf("Elemento removido: %d\n", pop(&p));
    exibir(&p);

    return 0;
}

```

Fila Circular em C (Array Estático)

A fila circular resolve o problema de **desperdício de espaço em arrays estáticos**, utilizando o operador módulo (%) para fazer com que os índices "voltem ao início" quando atingirem o limite do vetor.

```

# include <stdio.h>
# include <stdbool.h>

# define TAMANHO 5

```



```
typedef struct {
    int itens[TAMANHO];
    int inicio;
    int fim;
    int quantidade;
} FilaCircular;

void inicializarFila(FilaCircular *f) {
    f->inicio = 0;
    f->fim = -1;
    f->quantidade = 0;
}

bool estaCheia(FilaCircular *f) {
    return f->quantidade == TAMANHO;
}

bool estaVazia(FilaCircular *f) {
    return f->quantidade == 0;
}

// Inserção (enqueue)
void enqueue(FilaCircular *f, int valor) {
    if (estaCheia(f)) {
        printf("Erro: Fila Cheia!\n");
        return;
    }
    // Incremento circular do índice de fim
    f->fim = (f->fim + 1) % TAMANHO;
    f->itens[f->fim] = valor;
    f->quantidade++;
    printf("Inserido %d na fila.\n", valor);
}

// Remoção (dequeue)
int dequeue(FilaCircular *f) {
    if (estaVazia(f)) {
        printf("Erro: Fila Vazia!\n");
        return -1;
    }
    int valor = f->itens[f->inicio];
    // Incremento circular do índice de início
```

```
f->inicio = (f->inicio + 1) % TAMANHO;
f->quantidade--;
return valor;
}

void exibir(FilaCircular *f) {
    if (estaVazia(f)) {
        printf("Fila Vazia.\n");
        return;
    }
    printf("Fila atual: ");
    for (int i = 0; i < f->quantidade; i++) {
        int idx = (f->inicio + i) % TAMANHO;
        printf("%d ", f->itens[idx]);
    }
    printf("\n");
}

int main() {
    FilaCircular f;
    inicializarFila(&f);

    enqueue(&f, 10);
    enqueue(&f, 20);
    enqueue(&f, 30);
    enqueue(&f, 40);

    exibir(&f);

    printf("Removido: %d\n", dequeue(&f));
    printf("Removido: %d\n", dequeue(&f));

    exibir(&f);

    // Testando o comportamento circular
    enqueue(&f, 50);
    enqueue(&f, 60);
    enqueue(&f, 70); // Preenche o espaço liberado no início

    exibir(&f);

    return 0;
}
```

```
}
```